To Type Or Not To Type? An Empirical Follow-up Comparing Bug-Related Software Quality Outcomes of JavaScript and TypeScript Repositories on GitHub

Michael Andrews
Department of Software Engineering, Golisano College of Information Science
Rochester Institute of Technology
Rochester, NY, United States of America
mja6332@rit.edu

***Abstract*—Developers often disagree about the merits of static typing, citing the tradeoff between safety and verbosity. Developers on the positive side argue that typing helps prevent avoidable errors and make large codebases easier to maintain, while the critics argue that typing can add unnecessary overhead without guaranteeing better quality. This study takes a look at whether we can find any answers to that debate in observable artifacts from public GitHub repositories. I compared JavaScript and TypeScript repositories using two repository-level metrics as proxies for software quality: bug-fix commit ratio and median bug-related issue resolution time. The study follows a reproducible repository-mining workflow over a fixed study window from January 1, 2024, through December 31, 2025. Candidate repositories were discovered, filtered, screened for quality and maturity signals, enriched with language and activity metadata, and sampled from an eligible pool. The final sample contained 60 repositories, evenly split between JavaScript and TypeScript and balanced across activity strata. Across this sample, 102,519 commits and 76,308 closed issues were collected and classified using explicit rule-based criteria. TypeScript repositories showed a higher mean bug-fix commit ratio than JavaScript repositories and longer median bug-related issue resolution times at the language-group level. These results tell an interesting story, and subvert a common expectation, but did not ultimately show a causal effect of static typing. Instead the results, supported by prior work on the subject, suggest that language choice, project scale, bug maintenance practices, and issue-tracking behavior interact in ways that make it hard to analyze software quality as a product of just programming language choice alone.**

I. Introduction

Software quality is a critical domain within the broader discipline of software engineering. As AI-powered automation tools are accelerating code generation and construction, software quality is being approached more and more as a proactive design paradigm in the earlier stages of the software development lifecycle. With the field evolving at such a rapid pace, this "Shifting left" phenomenon means that more focus is going into preventing bugs instead of fixing them later. Therefore, furthering the research of which early development choices are associated with better software quality outcomes is integral to supporting reliable and more efficient software development.

One of the earliest development choices that receives exceptional amounts of attention is programming language choice. This foundational decision impacts nearly every aspect of software development. Different languages involve different tradeoffs in development speed, maintainability, tooling, and error prevention, and one major distinction among them is whether they use static typing.

Whether a language is statically typed or not determines how it enforces variables, functions, and objects to adhere to their assigned types. Specifically, statically typed languages add checks at compile time to ensure that the program will not compile unless typing is adhered to.

Depending on which developers you ask, this is either needless boilerplate that only serves to bog down the development process, or is a crucial safeguard against human error that helps keep programs readable and consistent. However, while these sentiments are often strongly held, they also are frequently based on personal preference alone, rather than any direct empirical evidence. Prior empirical work has attempted to move this debate beyond anecdote by studying associations between programming language features, type systems, and software quality outcomes [2], [3]. However, the divide persists among developers.

This study attempts to address that gap by examining whether repositories that use static typing exhibit different bug-related patterns than repositories that do not. Specifically, it compares two software quality-related metrics across repositories primarily written in JavaScript or TypeScript: bug-fix commit ratio and median bug-related issue resolution time. These metrics were selected because they provide concrete, repository-level indicators of bug-related development activity. JavaScript and TypeScript are especially useful for this comparison because they share the same broader ecosystem while differing in their use of static typing. This makes them a suitable pair for exploring whether static typing is associated with observable differences in bug-related outcomes.

This work is best described as a small-scale empirical follow-up to Bogner and Merkel's JavaScript/TypeScript repository-mining study [1]. It does not attempt to establish causal claims, but instead addresses two research questions:

RQ1, whether language choice is associated with bug-fix commit ratio during a fixed time window, and **RQ2**, whether language choice is associated with median bug-related issue resolution time during that same window. To answer these questions, I mined public GitHub repositories, applied multiple layers of filtering and eligibility screening, then drew a final stratified sample of 60 repositories, evenly split between JavaScript and TypeScript and stratified by repository activity. Commit and issue data were then collected, classified using explicit text and label-based rules, and aggregated into descriptive repository-level metrics.

II. Related Work

Prior research has already been done examining both the relationship between programming language choice and software quality and the methodological challenges of mining GitHub repositories. Bogner and Merkel provide the most directly relevant prior work for this study [1]. They compared JavaScript and TypeScript repositories on several software quality metrics, including code smells, cognitive complexity, bug-fix commit ratio, and bug issue resolution time. Their results suggested that TypeScript applications showed stronger results for code quality and understandability, but the bug-related outcomes were less clear-cut. In particular, TypeScript projects did not show clearly lower bug proneness or faster bug resolution. This ambiguity motivates the present study, which narrows the scope to bug-fix commit ratio and bug-related issue resolution time.

Broader empirical work has also examined whether programming language features are associated with software quality. Ray et al. conducted a large-scale GitHub study across multiple programming languages and found that language design features had a statistically noticeable but modest relationship with defects [2]. Importantly, they also emphasized that process factors such as project size, team size, and project history can eclipse language-level effects.

Other work has focused more directly on static typing in JavaScript. Gao, Bird, and Barr studied whether static type systems such as TypeScript and Flow could detect real JavaScript bugs before they were fixed [3]. Their findings support the idea that static typing can catch some bugs, but also show that its coverage is limited. This is relevant because this study began from a similar motivation, and the results, while different, share a similar sentiment.

Finally, this study follows prior warnings about mining GitHub data. Kalliamvakou et al. showed that GitHub is a rich but perilous source for empirical software engineering research, since repositories vary widely in activity, purpose, completeness, and use of GitHub features [4]. Their work supports a need for explicit filtering, quality screening, eligibility checks, and careful reasoning about commit and issue data. This study takes that advice to heart, utilizing a multi-staged pipeline in efforts to reduce some of those risks before drawing a final sample.

This study builds on the comparisons of Bogner and Merkel [1] as a follow-up. Instead of re-examining all of the quality metrics in their earlier work, it narrows the scope to the two bug-related outcomes that appeared to produce less certain results.

III. Methodology

A. Research Questions

This study addresses two research questions:

- **RQ1:** Among active public GitHub repositories whose primary language is JavaScript or TypeScript, is programming language associated with bug-fix commit ratio during a fixed study window?
- **RQ2:** Among those same repositories, is programming language associated with median bug-related issue resolution time during that same study window?

These questions were chosen because they focus on two concrete bug-related outcomes that can be measured from commit history and closed issue records. In theory, if one language showed a consistent association with either metric, that could motivate further investigation into whether typing-related differences are involved. However, such causal claims are not made in this paper. These two questions also follow up on the future research outlined in prior work.

B. Data Source and Candidate Selection

This study uses public GitHub repositories as its data source. GitHub was chosen because it provides easy public access to repo data, language stats, commit history, and issue labels. The study window was a fixed range from January 1st, 2024 through December 31st, 2025. All of the repository metrics gathered and aggregated were collected and computed from within this window of time.

The pipeline used both GitHub REST routes and GraphQL API data for differing levels of querying control. Repository discovery and basic metadata were gathered from GitHub repository search results. Candidates were enriched with GitHub language statistics for language threshold checks, main branch commit counts for recent activity levels, and GraphQL cursor pagination over closed issues for eligibility. Raw commit and issue collection then gathered the detailed records needed to classify each commit and issue, and aggregate the data into the desired repository-level metrics.

The initial discovery stage produced 944 raw candidate repositories. 444 of which were JavaScript and 500 were TypeScript repositories. These candidates were then passed through multiple filtering and screening stages before final sampling and stratification.

C. Candidate Discovery and Basic Filtering

Because GitHub repositories vary widely in purpose, activity, and completeness, the pipeline used explicit filtering and quality screening before sampling [4]. Candidate repositories were discovered separately for JavaScript and TypeScript. The discovery configuration requested five pages per language with 100 repositories per page, sorted by stars in descending order. This gave me a very broad starting set of visible public repositories in each language group that I could further whittle down through filtering and screening for the final sample.

After the discovery stage, the basic filtering stage removed repos that were clearly unsuitable based on metadata available from the repository search. This stage did not yet enforce the goal of the 70% language threshold for language majority. This was to save on API calls and tokens, instead this was deferred to a later enrichment step using GitHub language statistics. Instead, basic filtering removed repositories that were archived or did not have issues enabled since these were easy disqualifiers. After this stage 893 repositories remained: 409 JavaScript and 484 TypeScript. The logs recorded 51 exclusions, including 39 archived repositories and 12 repositories with issues disabled.

D. Quality and Maturity Screening

After the basic filtering stage, the repositories were passed through a software quality screening. The goal here was to reduce the chance that toy repositories, inactive, or weakly maintained repositories would enter the sampling pool and affect the final results.

The quality screen used a couple of repository-level signals. It considered recent maintenance activity, issue usage, and basic software engineering workflow indicators such as test-related file paths, CI/CD configuration files, and community contribution guidelines. Repositories were scored on whether they had these indicators of quality and recent activity, as well as a last-pushed window of 365 days. Repositories that passed this stage were kept and prepared for enrichment.

The quality screen evaluated 893 repositories. Of these, 849 passed, 44 failed. By language, 368 JavaScript repositories and 481 TypeScript repositories passed the quality check.

E. Candidate Enrichment and Pre-Sampling Eligibility.

The enrichment stage collected the concrete data needed before sampling. For each repository, the pipeline collected:

- target-language composition
- default-branch commit count during the study window
- closed issue count during the study window

The target-language share was computed from GitHub language statistics. The default branch commit count was collected from GitHub GraphQL commit history data. And Closed issue count was collected using GraphQL cursor pagination over the repository issue connection and counted issues whose closedAt timestamp fell within the study window.

Once Enrichment was complete, the pre-sampling research-question eligibility screen applied four checks. A given repository needed to satisfy a 70% target language composition threshold, have at least 50 default branch commits and 5 closed issues in the study window for recent activity, and have passed the quality screening.

The enrichment stage processed all 849 quality-screened repositories. Of these, 641 passed the language threshold, 600 passed the commit threshold, and 724 passed the closed-issue threshold. The final pre-sampling eligible pool contained 438 repositories: 129 JavaScript and 309 TypeScript repositories.

F. Sampling Strategy and Sample Size

The final sample was drawn from the pre-sampling eligible candidate pool. The methodology here was to start my search broad enough that after such stringent filtering, quality-screening, and eligibility phases, there would still be more than enough repositories left to sample from evenly. It also ensured that before any heavy commit or issue collection has begun, that all of the repositories being mined were fully eligible candidates to answer my research questions.

The target sample size was 60 repositories, split evenly between JavaScript and TypeScript. This produced 30 JavaScript and 30 TypeScript repositories. The sample was also stratified by activity levels referencing the default branch commit count within the study window. Within each strata, the eligible repositories were divided into high, medium, and low recent activity strata using terciles. The final sample selected 10 repositories per language from each activity stratum.

This sampling strategy was intended to avoid comparing very active repositories in one language against low-activity repositories in the other. This is a relevant metric because both research questions depend on commit and issue behavior. A repository with more development activity naturally will have more opportunities for bug-fix commits and closed issues. Balancing by language and activity therefore made the comparisons more sensible.

The final sample contained 60 repositories total, with exactly 30 JavaScript and 30 TypeScript repositories. Each language group included 10 high activity, 10 medium activity, and 10 low activity repositories.

G. Raw Data Collection

After sampling, the remaining stages utilized the sampled repositories as the active input. Commit collection gathered raw commit records for each sampled repository during the time window. Issue collection gathered closed issues during the same time window.

Raw data collection succeeded for all 60 sampled repositories. The pipeline collected 102,519 commits and 76,308 closed issues. The logs reported 0 failures, and 0 missing commits or issues.

H. Classification Rules

Classification was where the major important metrics from the raw data were collected and computed. Commit classification identified bug-fix commits from commit messages. The classifier required there to be bug-fix oriented language, including a required ‘fix’ term and then additional related terms such as ‘bug’, error’, defect’, crash’, or ‘issue’. For each repository, the classifier counted total commits and commits that were classified as bug-fix commits.

Issue classification identified bug-related issues using issue labels and issue text. The issue classifier used ‘bug’ as a label signal, and text terms such as ‘bug’, error’, defect’, and ‘failure’. For each issue classified as bug-related, the pipeline used the issue creation and closed timestamps to compute resolution time.

The final classification stage processed all the sampled repositories. It classified 102,519 commits, of which 2,596 were positive bug-fix classifiers. It also classified 76,308 issues, of which 42,552 were positive bug-related issue classifiers.

I. Metrics and Analysis Approach

For **RQ1**, the primary metric per repository was bug-fix commit ratio:

- Bug-fix commit ratio = bug-fix commit count / total commits in the study window

This metric was computed once per repository. The main quality intent here is to give a proxy for the proportion of development activity that involved bug-fixing.

For **RQ2**, the primary metric was median bug-related issue resolution time. For each issue classified as a bug related issue, the pipeline computed the elapsed time between issue creation and issue closing. The median of those issue durations was then used as the metric per repository. The median was chosen because resolution time can be very skewed with very short and very long running issues.

The analysis is purely descriptive. Each repo level metric was grouped by language and then summarized by count, mean, median, minimum, maximum, standard deviation, and interquartile range. No causal claims were made in this analysis, keeping the associations modest.

J. Tools and Reproducibility

The study was implemented as a reproducible data pipeline. The major scripts were:

- *scripts/collect/discover_candidates.py* for repository discovery
- *scripts/filter/filter_candidates.py* for basic filtering
- *scripts/filter/screen_final_candidates.*py for quality screening
- *scripts/collect/enrich_final_candidates.py* for language, commit, and closed-issue enrichment
- *scripts/filter/screen_final_eligibility.py* for pre-sampling eligibility
- *scripts/sample/draw_final_sample.py* for stratified final sampling
- *scripts/collect/fetch_commits.py* and scripts/collect/fetch_issues.py for raw data collection
- *scripts/collect/audit_final_collection.py* for collection completeness auditing
- *scripts/classify/classify_commits.py* and *scripts/classify/classify_issues.py* for rule-based classification
- *scripts/aggregate/compute_repo_metrics.py* for repository-level aggregation
- *scripts/analyze/generate_descriptive_outputs.py* for summary tables and figures

The study configuration was stored in *config/study_config.yaml*, including the study time window, the random seed used for sampling, languages, thresholds, sample size, and stratification settings. Keyword rules were stored in *config/keywords.yaml*. Each pipeline stage wrote intermediate artifacts, summaries, and logs so that the final results can be traced back through discovery, filtering, eligibility, sampling, collection, classification, and aggregation.

IV. Evaluation

This section presents the descriptive results for the final sample. The evaluation uses metrics computed from the final sample of 60 repositories. All 60 repositories contributed both commit and issue data. A collection audit was conducted and reported all 60 repositories completed collection successfully with 0 missing commit files, 0 missing issue files, and 0 collection failures. Therefore, each of the 60 sampled repositories contributed data to the final results.

Across the final sample, the pipeline classified 102,519 commits and 76,308 closed issues. Of the commit records, 2,596 were classified as bug-fix commits. Of the issue records, 42,552 were classified as bug-related issues. The following sections report the results for each research question

A. RQ1: Bug-Fix Commit Ratio

RQ1 asks whether programming language choice is associated with bug-fix commit ratio. For each repository, bug-fix commit ratio was calculated as the number of commits classified as bug-fix commits divided by the total number of commits mined in the study window.

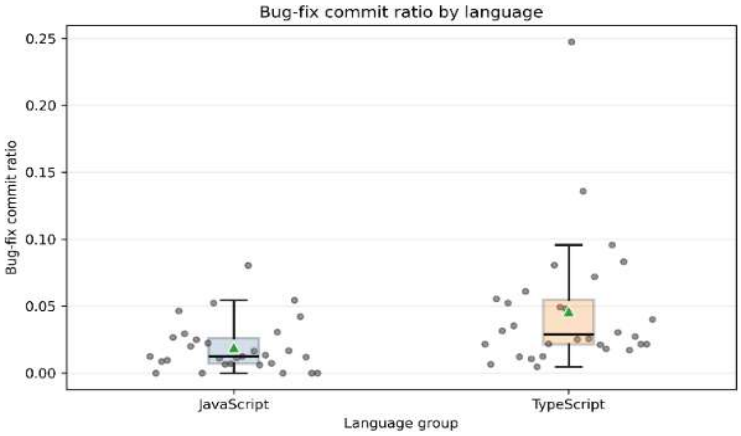

Fig. 1. Repository-level bug-fix commit ratio by language group. Each point represents one sampled repository. Boxplots show mean, median, interquartile range, and the distribution spread.

Table I. RQ1 Bug-Fix Commit Metrics by Language

Language	Repos	Mean bug-fix ratio	Median bug-fix ratio	Mean bug-fix commits	Median bug-fix commits	Mean total commits
JavaScript	30	0.0194	0.0126	13.7	3.5	945.6
TypeScript	30	0.0462	0.0289	72.8	21.0	2471.7

The descriptive stats in figure 1 show that TypeScript repositories had a higher bug-fix commit ratio than JavaScript

repositories in this sample. The mean bug-fix commit ratio was 0.0462 for TypeScript and 0.0194 for JavaScript. The same pattern appears in the medians: 0.0289 for TypeScript and 0.0126 for JavaScript. This suggests that the difference in these values might not be because of a few high-value repositories skewing the data, although the TypeScript group still does contain a much wider range of values.

The supporting commit count metrics show that TypeScript repositories also had more bug-fix commits in the absolute sense. The mean TypeScript repository had 72.8 bug-fix commits, compared with 13.7 for JavaScript. TypeScript repositories also had more total commits on average.

When the counts are aggregated across the repositories, JavaScript repositories contributed 28,369 commits and 411 bug-fix commits, while TypeScript repositories contributed 74,150 commits and 2,185 bug-fix commits. This corresponds to bug-fix commit shares of approximately 1.45% for JavaScript and 2.95% for TypeScript.

B. RQ2: Bug-Related Issue Resolution Time

RQ2 asks whether programming language is associated with median bug-related issue resolution time. Figure 2 shows the repository-level median resolution time for bug-related issues. The figure shows that both languages are very skewed. Many repositories have low median resolution times, while a small number of repositories have much longer median resolution times. This skew is especially apparent in the TypeScript group.

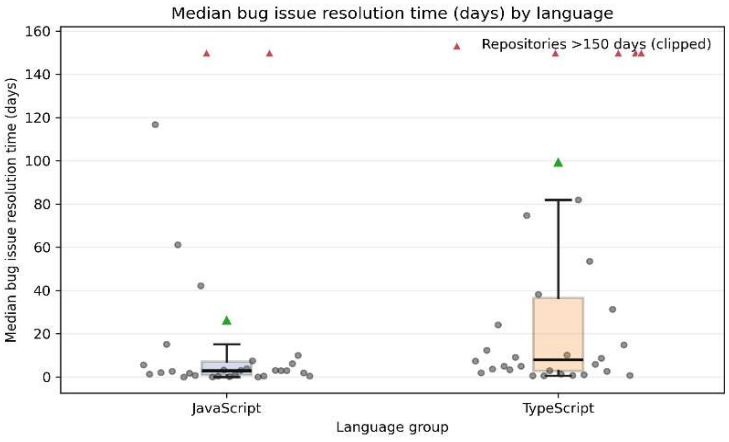

Fig. 2. Repository-level median bug-related issue resolution time by language group. Each gray dot represents one repository. Red triangles indicate repositories with median resolution times greater than 150 days. These points are visually clipped at 150 days for readability. Group summary values are reported in Table II.

Table II. RQ2 Bug-Related Issue Metrics by Language

Lang	Repos	Mean - median resolution days	Median -median resolution days	Mean bug-related issues	Median bug-related issues	Mean closed issues
JS	30	26.53	3.01	139.2	71.5	257.2
TS	30	99.54	7.98	1279.2	178.0	2286.4

Table II shows that TypeScript repositories had longer bug-related issue resolution times than JavaScript repositories in this sample. The mean repository-level median resolution time was 99.54 days for TypeScript and 26.53 days for JavaScript. The median of those repository-level medians was also higher for TypeScript: 7.98 days compared with 3.01 days for JavaScript.

The difference between the mean and median values is important. Both language groups contain large outlier behavior, but the TypeScript group has especially large outliers. For example, the maximum repository-level median resolution time was 913.89 days for TypeScript, compared with 292.07 days for JavaScript. This indicates that the mean resolution time is sensitive to a small number of repositories with very long-lived bug-related issues. The median values are therefore more stable for describing the typical repository.

Issue volume also differed substantially between the two language groups. TypeScript repositories had a mean of 1279.2 bug-related issues, while JavaScript repositories had a mean of 139.2. The TypeScript group also had a much larger mean number of closed issues considered. This suggests that some of the RQ2 differences may reflect project scale and issue-tracking intensity, not language alone.

C. Activity-Stratum Observations

Table III. RQ Metrics by Activity Strata

Metric	Language	High	Medium	Low
Mean bug-fix ratio	JavaScript	0.0178	0.0226	0.0176
Mean bug-fix ratio	TypeScript	0.0292	0.0718	0.0377
Median issue resolution days	JavaScript	2.36	3.19	3.01
Median issue resolution days	TypeScript	6.20	2.15	23.08

Because the final sample was balanced across high, medium, and low-activity strata, I also inspected whether the language-level pattern was concentrated in only one activity group. TypeScript had higher mean bug-fix commit ratios than JavaScript in all three strata. The largest difference appeared in the medium-activity stratum, where TypeScript repositories had a mean bug-fix ratio of 0.0718 compared with 0.0226 for JavaScript. For bug-related issue resolution time, the pattern was less uniform. TypeScript had longer median resolution times in the high and low activity strata, while JavaScript was higher in the medium activity stratum. This reinforces the fact that RQ2 was much more sensitive to skewing than RQ1.

D. Summary of Results

Overall, the descriptive results show two main patterns. First, TypeScript repositories in the final sample had higher bug-fix commit ratios than JavaScript repositories. This was visible in both the mean and median repository-level ratios, and it was also visible in the generated distribution plot. Second, TypeScript repositories had longer bug-related issue resolution times, but this result was more strongly affected by skew and

outliers. Several TypeScript repositories had very large issue volumes and long median resolution times, which increased the group mean.

These findings should be interpreted as descriptive associations rather than causal effects. The results do not show that TypeScript causes more bug fixes or slower issue resolution. Instead, they show that in this sampled set of public GitHub repositories, TypeScript repositories exhibited more bug-fix commit activity and longer bug-related issue resolution times than JavaScript repositories during the study window, and that's it.

V. Discussion

In the introduction I touched on the arguments between static typing developers and dynamic typing developers. The arguments on the pro-typing side highlight that static typing helps to guard against typing related bugs and more subtle error states that can arise due to human error. The compiler enforces these rules, so human error cannot cause unforced errors. On the contrary, the pro dynamic typing developers argue that static typing is extra overhead that adds unnecessarily verbose, boilerplate code. Typically, they argue that the benefits it offers do not outweigh the negatives.

The results of this study complicate the expectations around this debate. I personally expected TypeScript repositories to show clearer evidence of lower bug-related maintenance artifacts, based on the merits of the positive argument. I also think this expectation was reasonable because prior work has shown that static typing systems can be used to detect some real bugs, while not all of them [3]. However, the descriptive results did not necessarily support my expectation in a straightforward way.

A. RQ1: Bug-Fix Commit Ratio Interpretations

For **RQ1**, TypeScript repositories showed a higher average bug-fix commit ratio than JavaScript repositories. The mean bug-fix commit ratio was 0.046 for TypeScript repositories compared with 0.019 for JavaScript repositories, and the median was 0.029 compared with 0.013. However, this does not mean that TypeScript causes more bugs, or that TypeScript projects are lower quality.

A possibly more reasonable interpretation is that TypeScript repositories in this sample may represent larger, more actively maintained, or more formally managed projects where bug-fix work is more visible in commit history. In other words, bug-fix commit ratio may not only capture defect frequency, but also speak to the maintenance discipline, commit-message conventions, and activity levels of the project.

This finding does run against the somewhat common intuition about TypeScript and static typing. Static typing is often discussed as a way to reduce certain types of errors, especially as applications and repositories get larger. However, the metric used here does not directly measure the number of defects in the codebase. It measures the proportion of commits that are dedicated to bug-fix work. A project with strong quality requirements or maintenance standards may naturally have more bug-fix commits because bugs might be more heavily tracked, labeled, and focused on in a more visible way.

This could mean that bug-fix commit ratio doesn't necessarily measure the frequency that bugs arise in development. It could instead be a more accurate proxy for certain development processes, requirements, and habits revolving around maintaining higher quality software. A Software project may have a lower bug-fix commit ratio not because there are less bugs, but because less developers are putting time and effort into fixing them. It could also simply mean that the commit histories of such repositories are simply less descriptive as well.

B. RQ2: Bug-Related Issue Resolution Time Interpretations

For **RQ2** TypeScript repositories also had longer median bug-related issue resolution times. The median repository-level resolution time was approximately 8.0 days for TypeScript repositories and 3.0 days for JavaScript repositories. The mean difference was larger, at about 99.5 days for TypeScript compared with 26.5 days for JavaScript, but this result was strongly influenced by outliers.

These Issue resolution results suggest that TypeScript repositories in the sample may include projects with larger issue backlogs and longer-lived bug reports. Again, I don't think I can attribute these results as a consequence of the programming language necessarily.

However, I do think it could potentially speak to the *types* of projects that adopt TypeScript over JavaScript. TypeScript is typically used in larger, more complex repositories, like the findings supported earlier. These repositories with heavier infrastructure, more in-depth tooling, and larger integration of developer tools may also naturally receive more issue reports. These issues may require more contributors and require a larger amount of coordination in part due to their increased complexity and scope.

C. General Interpretations

Taken together, the two research questions seem to suggest that TypeScript repositories in this sample were not simply "better" or "worse" than the JavaScript repositories. They appeared to have more bug-fix related activity as per RQ1, and issues that set out to fix bugs or errors on average took longer to resolve as per RQ2.

One possible interpretation is that the TypeScript projects in the final sample were simply more formally managed than the JavaScript sample. Another possibility is that adopting TypeScript correlates with greater project scale and complexity, which may increase both bug-fix activity and issue resolution time. These interpretations are consistent with prior evidence that process and project factors can dominate language-level effects in repository-mining studies [2].

The main take-away from these findings is that repository-mining metrics should be reasoned about carefully. A higher bug-fix commit ratio can sound like a correlation with higher bug count. However, it may also indicate a higher level of maintenance, documentation, and overall project maturity. A longer issue resolution time could suggest slower maintenance and development, but it may also reflect more complex problems and issues to be solved. Larger communities that contribute to a project or more stringent review and merge

practices could also explain these discrepancies. These metrics are useful data points, but they do not directly correlate to better or worse software quality.

Practically, the results shown suggest that programming language alone is not enough to draw conclusions about overall software quality. Many developers, me included, see statically typed languages as ways to improve general project maintainability as complexity scales. But there are also other very important elements to consider, such as the project size, the number of contributors, the issue management process, commit conventions, maintenance practices, and project requirements that can all shape data and software quality.

Overall, the results are interesting and a bit surprising because they resist the simple narrative that TypeScript repositories should automatically show cleaner bug-related outcomes. Instead, the observed associations point towards a more nuanced explanation, that requires consideration for multiple multifaceted variables. TypeScript repositories in the sample may be larger, more active, or more complex environments. The results provide a useful comparison, but only really function as a starting point for future study into the more complex nuanced elements that make up software quality.

VI. Threats to Validity

There are multiple threats to validity in this study. The results should be taken as purely descriptive data from a sampled set of GitHub repositories. There is no causal evidence that either JavaScript or TypeScript produce a better or worse software quality outcome.

A. Construct Validity

A major threat to construct validity is the way bug-related activity was measured. Bug-fix commits were identified using commit message classification rules, and bug-related issues were identified using issue labels and text-based rules. These signals are very practical for repository mining, but they are not perfect proxies for actual software bugs. Some bug fixes could use non-bug-related language, while some commits that mention bugs may not really represent an actual fix. In the same vein, issue labels and titles may depend heavily on individual repository or community practices. A repository with more consistent issue labeling may appear to have more bug-related issues than a repository with weaker labeling, even if the actual defect density is similar.

The issue-resolution metric Is also a construct validity risk itself. The metric measures the time between issue creation and issue closure for bug-related issues. However, issue closure does not always mean a bug was fixed, and as I discussed earlier, issue labels and titles can have a lot of variance. Issues may be closed as duplicates, stale reports, support questions, invalid reports, or because work was moved elsewhere. Also, a bug may be fixed before its issue is formally closed. For these reasons, median bug-related issue resolution time is probably better interpreted as a GitHub issue-lifecycle measure rather than a perfect proxy measure of engineering time.

B. Internal Validity

There are also internal validity threats. The comparison groups differ by primary language, but many other repository characteristics can influence the results. Project size, domain, age, contributor count, dependency complexity, and maintainer practices could all affect bug-fix commit ratios and issue resolution times. TypeScript repositories in the sample may include larger or more complex projects, which can help explain both the higher bug-fix activity and longer issue resolution times. Because this study does not take those factors into account, the differences in the results should be treated as associations at best, rather than causal evidence of a direct language effect. These limitations are consistent with broader concerns about mining GitHub data, including repository inactivity, inconsistent use of issues, and variation in repository purpose [4].

Repository process differences create another internal-validity threat. Some projects use generic commit messages or track bugs outside GitHub. Others use detailed labels and explicit bug-fix commit messages. These process differences can affect the measured outcomes independently of actual software quality. This is especially important because the study depends on repository artifacts rather than a direct observation of development work.

C. External Validity

External validity is limited by the sampling scope. The study only includes public GitHub repositories that passed my final eligibility screen, including language threshold, commit-count, issue-count, and quality/maturity requirements. These criteria improved the consistency of the final sample, but they also exclude smaller, less active, private, archived, or non-GitHub projects. As a result, the findings may not generalize to all JavaScript and TypeScript projects, to private codebases, or to projects using different workflows. The final sample size of 60 repositories also limits how broadly the results should be considered.

D. Conclusion Validity

Finally, conclusion validity is limited because the study uses descriptive analysis rather than actual formal hypothesis testing. The results show descriptive differences between the sampled JavaScript and TypeScript repositories, but they do not establish whether language choice itself explains them. In addition, several issue-resolution values were strongly affected by large outliers, which is why medians are more informative than the means for interpreting the results.

Overall, these threats do not invalidate the study, but they do limit the strength of the conclusions that can be drawn from it. The findings are descriptive in their comparison of bug-related repository-level signals, and do not properly serve as definitive evidence about the inherent quality or reliability of either language.

VII. Data and Code

The public repository for this study is available at: https://github.com/maj3-02/SWEN640-Research-Paper. It contains the final-study pipeline, configuration files, tests,

documentation, and the data artifacts used to support the results reported in this paper.

The repository is organized so that readers can inspect the results directly or rerun the workflow. The final sample is stored in data/interim/final_sample/final_sample.csv, the row-level classified data is stored under data/interim/final_sample/classified_commits/ and data/interim/final_sample/classified_issues/, and the processed repository metrics used for analysis are stored in data/processed/repo_metrics/final_sample/repository_metrics.csv. The final tables and figures are stored under results/final_sample/. The repository also includes a runbook, reproducibility notes, and a data dictionary to explain the major artifacts and execution order.

VIII. Conclusion

This study investigated whether JavaScript and TypeScript repositories differ in observable bug-related maintenance activity on GitHub. Using a reproducible software repository mining pipeline, the study selected a balanced sample of 60 repositories, collected commit and issue data from a fixed study window, classified bug-related commits and issues, and aggregated repository-level metrics for comparison.

The results did not directly support the simple expectation that TypeScript repositories would show clearly lower bug-related maintenance activity. In this sample, TypeScript repositories had a higher bug-fix commit ratio than JavaScript repositories and longer median bug-related issue resolution times. These findings should not be interpreted as evidence that TypeScript causes more defects or that JavaScript projects are inherently easier to maintain. Instead, they suggest that public repository metrics often reflect a combination of different variables, such as language choice, project size, development practices, issue-tracking behavior, and maintainer conventions.

The main justification for this study was to be a reproducible follow-up comparison of bug-related activity signals across JavaScript and TypeScript repositories. The findings were similar to prior work in that the association between programming language and bug-related activity isn't 100% clear. From the data gathered it seems that language-level expectations about maintainability do not always translate directly into easily detectable repository-level outcomes. Future work could expand the sample size, include additional languages, apply manual validation to the classification rules, control for project size and domain, and examine how TypeScript-specific practices such as type coverage or use of the any type relate to bug-related maintenance outcomes.

Overall, the study provides descriptive evidence that the relationship between programming language and software maintenance quality is more complex than a simple typed-versus-untyped comparison. Repository-mining studies can help reveal these patterns, but their results must be interpreted carefully and in context. And unfortunately for me, this study brought us no closer to finally resolving the typing debate between developers.

References

[1] J. Bogner and M. Merkel, "To type or not to type? A systematic comparison of the software quality of JavaScript and TypeScript applications on GitHub," in Proc. 19th Int. Conf. Mining Software Repositories (MSR), 2022, pp. 658–669, doi: 10.1145/3524842.3528454.

[2] B. Ray, D. Posnett, V. Filkov, and P. T. Devanbu, "A large scale study of programming languages and code quality in GitHub," in Proc. 22nd ACM SIGSOFT Int. Symp. Foundations of Software Engineering (FSE), 2014, pp. 155–165, doi: 10.1145/2635868.2635922.

[3] Z. Gao, C. Bird, and E. T. Barr, "To type or not to type: Quantifying detectable bugs in JavaScript," in Proc. IEEE/ACM 39th Int. Conf. Software Engineering (ICSE), 2017, pp. 758–769, doi: 10.1109/ICSE.2017.75.

[4] E. Kalliamvakou, G. Gousios, K. Blincoe, L. Singer, D. M. German, and D. Damian, "The promises and perils of mining GitHub," in Proc. 11th Working Conf. Mining Software Repositories (MSR), 2014, pp. 92–101, doi: 10.1145/2597073.2597074.